

Setting up a Hauppauge USB TV Tuner in Arch Linux on a RPi4

Published on Jul 7, 2020

linux

Let's take a look at how to set up a USB TV Tuner with Linux using `w_scan2`, `dvbv5-zap`, and `ffmpeg`.

Prerequisites

Hardware

You probably already have some hardware in mind but for reference, I am using a Hauppauge tuner originally designed for Xbox but it works just as well with Linux. My corresponding `lsusb` line is `ID 2040:b123 Hauppauge`. I'm using this with a Raspberry Pi 4, 4GB but nothing here is particular to the Pi so it shouldn't make any difference. I have installed 32-bit Arch Linux on it as shown in an earlier post.

Permissions

Most likely, your video device is already moved to a `video` group for security. But your user may or may not be part of that group already. To add yourself to the group that can access your video device, check

I use `sudo` rather than `su` in this tutorial, but which one is appropriate depends on your installation

```
# Check that the devices are there
test -d /dev/dvb/adapter0 || echo "DVB devices are missing."
# Read this to see what the group is for your adapter. Usually it's "video"
ls -lh /dev/dvb/adapter0/*
# Add yourself to "video" or whichever group the adapter is in
sudo usermod -aG video $USER
```

At this point you should logout/login or restart in order for group settings to take effect

Software

You need `w_scan2` to search for channels in your area. There are other scanning utilities, like `dvbv5-scan` but they require you already have an initial scan file, which is difficult to find or create without using `w_scan2`, and `w_scan2` can already do the rest of the scan for you too. So you might as well use `onlyw_scan2`

```
# Build requirements
sudo pacman -S --needed base-devel git

git clone https://github.com/stefantalpalaru/w_scan2.git
cd w_scan2
./autogen.sh
./configure
make -j `nproc`
```

Scan for Channels

Actually scanning for channels is not that hard once you have the right package. Feel free to poke around with the array of options `w_scan2` offers but I found

most of them have reasonable defaults for over the air TV. YMMV, especially if you are trying to use cable or satellite. I doubt whether these cards will be of any use in those cases because they are usually encrypted. At any rate, your command will probably look something like the following:

```
# Still in the w_scan2 repository, run the scan  
./w_scan2 -c US -M | sed 's/VSB-8/8VSB/g' | tee > ~/channels.conf
```

Side note on modulation errors

This is where things derailed for me. I kept getting errors that looked like this:

```
ERROR parameter MODULATION invalid: VSB_8 while parsing line 1 of /home/
```

In my case I had to replace instances of **VSB-8** with **8VSB**, which I only found by combing through the **dvbv5-zap** source code looking for what caused the error I saw. It seems the modulations are hard-coded and disagree on what exactly the names should be. It was hard to find but easy to fix, which you can see I did with **sed**. If you are getting a similar error you might need to see if you are affected by a similar problem.

Recording

The old way

mencoder can copy the recording for you, but I doubt whether you will like the result. The issue is that **mencoder** is trying to do too much by packing and unpacking the data, which:

- Wastes CPU and could cause you to skip some frames unnecessarily
- Propagates reception errors
- Causes problems with A/V sync

There may be a combination of `-mc _`, `-noskip` and others that can resolve the AV sync issues, but even though it shows up a lot in tutorials, it's probably not necessary for digital tuners these days. It's already an MPEG stream so you can just copy it literally straight to disk and sidestep this whole problem.

Don't bother copy-pasting this one

```
mencoder dvb://CHANNELNAME -of mpeg -ovc copy -oac copy -idx -mc 0 -o 0l
```

The new way

`dvbv5-zap` supports concatenating the tuner output to a file, which is probably the simplest and most sane thing to do. In it's simplest form, it would look something like this:

```
dvbv5-zap \
  -I zap \
  -c "~/channels.conf" \
  -C US \
  -r \
  -o "SERIES_NAME/`date -Isec`.mpg" \
  -t 3595 \
  CHANNEL_NAME
```

Option	Purpose
<code>-I zap -c ~/channels.conf</code>	reads your channel list in <code>zap</code> format
<code>-C US</code>	uses US frequencies (this may make no difference since you scanned already)
<code>-r -o yadayada.mpg</code>	records the tuner output to <code>yadayada.mpg</code>
<code>date -Isec</code>	just creates a timestamp to keep from clobbering old recordings
<code>-t 3595</code>	records for five seconds less than two hours, to avoid overlapping with another recording

Option	Purpose
<code>SERIES_NAME</code>	is whatever you want it to be (but the folder must already exist before recording)
<code>CHANNEL_NAME</code>	is the name mentioned in the far left column of your channels list

Encoding later

Now that your video has already been received, you can take your time encoding it rather than trying to do it on the spot without hiccups. This is quite easy to do and I recommend doing it with `ffmpeg` and open codecs, specifically VP9, Opus, and MKV/WEBM. As a non-free alternative you may choose `h264` and `aac` in `mp4`.

If you use `schedtool` later on, then this process would be good to place on Batch or Idle priority using `schedtool -D -n 19 -e ffmpeg ...`

```
# set $source and $target to the input and output filenames first
```

```
ffmpeg -y -i "$source" -c:v libvpx-vp9 -pass 1 -b:v 300K -crf 34 -speed
-c:a libopus -b:a 64k -ac 6 -mapping_family 1 -f webm /dev/null
```

```
ffmpeg -i "$source" -c:v libvpx-vp9 -pass 2 -b:v 300K -crf 34 -speed 1 \
-auto-alt-ref 1 -lag-in-frames 25 \
-c:a libopus -b:a 64k -ac 6 -mapping_family 1 -f webm "$target"
```

Option	Purpose
<code>-y</code>	Overwrite output if necessary
<code>-i "\$source"</code>	Input file
<code>-c:v libvpx-vp9</code>	VP9 video codec
<code>-pass 1</code>	First of a two pass encoding (for better quality)
<code>-b:v 750K -crf 34</code>	Target about 750Kbit/s - change this to your own liking.
<code>-auto-alt-ref 1 -lag-in-frames 25</code>	Enable more advanced encoding

Option	Purpose
-c:a libopus	Use Opus for audio
-b:a 64k	Target 64k bitrate for opus
-ac 6 -mapping_family 1	Use six channels for audio, which is mostly to fix oddball channel setups
-f webm	Store it in a webm (Matroska) container

The bitrates shown are appropriate for medium-quality SD but you can find bitrate recommendations from Google as well

Extras

Clock

It's a good plan to keep your clock synced if you want to schedule recording, so I recommend installing `ntpd` to periodically sync your system clock, which is pretty easy to do

```
sudo pacman -S openntp
sudo systemctl enable openntp
sudo systemctl start openntp
```

Priority

Newer Linux kernels support soft realtime scheduling, which you can use with `schedtool`.

```
sudo pacman -S schedtool
```

```
# You must run schedtool as root to use the realtime scheduler,
# but then you should move back to your own user as soon as possible
sudo schedtool -R -p 10 -e \
  sudo -u YOUR_ORIGINAL_USER_HERE \
    whichever recording command you choose
```

Putting it together

Setting up your installation will require your full attention but the recording part can be pretty easily automated with a script like the following.

```
set -e
script_home="\`dirname $0`"
video_user=$1
output_folder="$2"
channel="$3"
timeout="$4"

if test `whoami` -ne "root"; then
    echo "This script needs to run as root to use realtime scheduling."
    exit 1
fi

if ! test -f "$script_home/channel.list"; then
    echo "Channel list not found at $script_home/channel.list"
    exit 1
fi

sudo -u $video_user mkdir -p "$output_folder"

schedtool -R -p 10 -e \
    sudo -u $video_user \
        dvbv5-zap \
            -I zap \
            -C US \
            -c "$script_home/channel.list" \
            -r \
            -t $timeout \
            -o "$output_folder/`date -Isec`.mpg" \
            "$channel"
```